Linux Security Governance Practical Laboratory

GRC102 Practical Lab 8

Item	Details
Student	Adediran Eunice Adebukola
Registration number	C11-26-CGRCE-17 595
Course	GRC102 Information Security Governance
Assessment	Security governance simulation and control evaluation
Date	11 October 2026

This report documents the creation of a controlled Docker environment, vulnerability assessment, governance-failure analysis, simulated control implementation, verification, and executive security reporting. The exercise reproduced the central governance conditions associated with the Equifax breach: a known unpatched application vulnerability, weak database credentials, ineffective segmentation and inadequate monitoring.

The main conclusion is that policies and recorded remediation status are not sufficient evidence of control effectiveness. Although the supplied control scripts recorded successful simulated remediation, the technical retest continued to identify the vulnerable service and the controlled attack still succeeded. This gap is treated as a verification finding rather than concealed.

Executive Summary

The laboratory environment contained an Apache Struts web service, a MySQL database and a monitoring container. The web service responded successfully on port 8080. A purpose-built scanner identified one critical finding, one high finding and one inconclusive network-segmentation result. The controlled attack simulation then confirmed that CVE-2017-5638 could be detected in the assigned environment and recorded simulated access to three test customer records.

Governance analysis identified eight failures across patch management, incident detection, policy implementation, vulnerability management, metrics, security architecture, authentication and access control. The control phase introduced recorded patch, segmentation, database, monitoring and oversight controls. A defect in the supplied patch script overwrote earlier status values; the combined status was reconstructed from the three successful patch-operation outputs. Even after that correction, the scanner and attack retest remained successful because the supplied controls simulated remediation rather than changing the underlying images and credentials.

Area	Result	Governance meaning
Environment	Three containers operational; HTTP 200	Suitable controlled test environment
Initial scan	1 critical, 1 high, 1 unknown	Immediate remediation and validation required
Attack simulation	Successful; 3 test records affected	Exposure can translate into business impact
Failure analysis	8 governance failures	Weaknesses span people, process and technology
Control status	All simulated flags recorded as true	Implementation records exist
Technical retest	Critical/high findings and attack persisted	Controls were not technically effective

Scope and Method

The work followed the supplied practical instructions in four parts. Commands were executed only against the assigned local laboratory. Because the original cloud VM could not reach Docker Hub and the host is Apple Silicon, the containers were completed with Docker Desktop's Linux virtual machine and AMD64 emulation. The obsolete Struts tag in the instructions was replaced with the available Vulhub Struts 2.3.30 image. The application address was changed from the internal Docker hostname to 127.0.0.1:8080 for scripts executed on macOS.

- Evidence was captured as terminal output, JSON reports, Markdown reports, HTML dashboard output and PNG charts.
- The attack script used the benign detection mechanism supplied in the brief and did not target an external system.
- Results were preserved even where they contradicted the expected outcome.

Part 1 Setting Up the Simulated Environment

Environment configuration

The environment used a dedicated Docker network and three services: the vulnerable Struts web application, MySQL 5.7 with seeded test data and an Ubuntu monitoring container. Container status and the HTTP response demonstrated that the services were available before testing began.

```
Last login: Sun Oct 11 14:57:18 2026 from 192.168.252.1
ubuntu@grc102-week8:~$ docker load -i ~/mysql57-amd64.tar
cff044e18624: Loading layer [==================================================>]    145MB/145MB
7ff7abf4911b: Loading layer [==================================================>]  11.26kB/11.26kB
8b2952eb02aa: Loading layer [==================================================>]  2.383MB/2.383MB
d76a5f910f6b: Loading layer [==================================================>]  13.91MB/13.91MB
8527ccd6bd85: Loading layer [==================================================>]  7.168kB/7.168kB
4555572a6bb2: Loading layer [==================================================>]  3.072kB/3.072kB
0d9e9a9ce9e4: Loading layer [==================================================>]  79.38MB/79.38MB
532b66f4569d: Loading layer [==================================================>]  3.072kB/3.072kB
337ec6bae222: Loading layer [==================================================>]  278.8MB/278.8MB
73cb62467b8f: Loading layer [==================================================>]  17.41kB/17.41kB
441e16cac4fe: Loading layer [==================================================>]  1.536kB/1.536kB
Loaded image: mysql:5.7
ubuntu@grc102-week8:~$ docker images mysql
REPOSITORY   TAG       IMAGE ID       CREATED       SIZE
mysql        5.7       5107333e08a8   2 years ago   501MB
ubuntu@grc102-week8:~$ cd ~/security_governance_lab
ubuntu@grc102-week8:~/security_governance_lab$ DOCKER_DEFAULT_PLATFORM=linux/amd64 docker-compose up -d
Pulling monitoring (ubuntu:20.04)...
20.04: Pulling from library/ubuntu
ecd83b6c3544: Pull complete
Digest: sha256:8feb4d8ca5354def3d8fce243717141ce31e2c428701f6682bd2fafe15388214
Status: Downloaded newer image for ubuntu:20.04
Creating web_server        ... done
Creating database_server   ... done
Creating monitoring_server ... done
ubuntu@grc102-week8:~/security_governance_lab$ sleep 30

ubuntu@grc102-week8:~/security_governance_lab$ docker-compose ps
      Name                     Command               State                    Ports
-------------------------------------------------------------------------------------------------
database_server     docker-entrypoint.sh mysqld      Up      3306/tcp, 33060/tcp
monitoring_server   tail -f /dev/null                Up
web_server          /usr/local/bin/mvn-entrypo ...   Up      0.0.0.0:8080->8080/tcp,:::8080->8080/tcp
ubuntu@grc102-week8:~/security_governance_lab$ docker-compose ps | tee evidence/03-container-status.txt
      Name                     Command               State                    Ports
-------------------------------------------------------------------------------------------------
database_server     docker-entrypoint.sh mysqld      Up      3306/tcp, 33060/tcp
monitoring_server   tail -f /dev/null                Up
web_server          /usr/local/bin/mvn-entrypo ...   Up      0.0.0.0:8080->8080/tcp,:::8080->8080/tcp
ubuntu@grc102-week8:~/security_governance_lab$ docker-compose logs --tail=40 2>&1 | tee evidence/04-container-logs.txt
Attaching to monitoring_server, database_server, web_server
web_server    | qemu: uncaught target signal 11 (Segmentation fault) - core dumped
ubuntu@grc102-week8:~/security_governance_lab$
```

Figure 1 Docker services running and web service available

Vulnerability scan

The scanner reported three findings. The critical Struts result concerned CVE-2017-5638. The high result showed that MySQL accepted the configured default laboratory credentials. The network check returned Unknown because the script attempted to decode a MySQL binary protocol response as UTF-8. That result is therefore inconclusive rather than proof that segmentation was effective or ineffective.

Finding	Severity	Evidence	Recommended response
Apache Struts S2-045	Critical	Detection header returned	Upgrade beyond affected versions and retest
Default MySQL credentials	High	Root login succeeded	Replace defaults and restrict database access
Network segmentation	Unknown	UTF-8 decoding error	Validate connectivity with a protocol-appropriate test

```
(.venv) ➜  security_governance_lab python3 -m json.tool vulnerability_report_20261011_152843.json | tee evidence/07-vulnerability-report-readable.txt
{
    "scan_date": "2026-10-11T15:28:43.934620",
    "vulnerabilities": [
        {
            "vulnerable": true,
            "vulnerability": "Apache Struts2 S2-045 (CVE-2017-5638)",
            "severity": "Critical",
            "description": "Remote Code Execution vulnerability in Apache Struts2",
            "remediation": "Update to Struts 2.3.32 or Struts 2.5.10.1",
            "reference": "https://nvd.nist.gov/vuln/detail/CVE-2017-5638"
        },
        {
            "vulnerable": true,
            "vulnerability": "MySQL using default credentials",
            "severity": "High",
            "description": "MySQL 5.7.44 accessible with default credentials",
            "remediation": "Change default credentials and restrict access",
            "reference": "https://dev.mysql.com/doc/refman/5.7/en/security.html"
        },
        {
            "error": "'utf-8' codec can't decode byte 0xff in position 25: invalid start byte",
            "vulnerability": "Insufficient network segmentation",
            "severity": "Unknown",
            "description": "Error checking vulnerability",
            "remediation": "Manually verify the system",
            "reference": "https://www.cisecurity.org/controls/implement-network-segmentation"
        }
    ],
    "summary": {
        "total": 3,
        "critical": 1,
        "high": 1,
        "medium": 0,
        "low": 0,
        "unknown": 1
    }
}
(.venv) ➜  security_governance_lab ls -lh evidence
total 88
-rw-r--r--  1 macbook  staff   579B Oct 11 14:34 01-environment-baseline.txt
-rw-r--r--  1 macbook  staff   506B Oct 11 15:21 03-container-status-macos.txt
-rw-r--r--  1 macbook  staff   567B Oct 11 15:12 03-container-status.txt
-rw-r--r--  1 macbook  staff    10K Oct 11 15:21 04-container-logs-macos.txt
-rw-r--r--  1 macbook  staff   152B Oct 11 15:12 04-container-logs.txt
-rw-r--r--  1 macbook  staff   252B Oct 11 15:21 05-web-response.txt
-rw-r--r--  1 macbook  staff   179B Oct 11 15:28 06-vulnerability-scan-output.txt
-rw-r--r--  1 macbook  staff   1.4K Oct 11 15:30 07-vulnerability-report-readable.txt
-rw-r--r--  1 macbook  staff   1.2K Oct 11 15:29 vulnerability_report_20261011_152843.json
(.venv) ➜  security_governance_lab
```

Figure 2 Initial vulnerability report and preserved evidence

Governance baseline

The governance tracker was initialized with three policies, three controls, three risks, one incident, three metrics and one audit. Its dashboard showed one critical, one high and one medium risk. Patch compliance was 85% against a 95% target, while vulnerability remediation time was twelve days against a seven-day target. These values show that governance mechanisms existed but were not producing timely remediation.

Part 2 Analyzing Security Governance Failures

Controlled attack simulation

The controlled script confirmed the vulnerable Struts condition and simulated data exfiltration from the customer database. The resulting attack log recorded a successful event affecting three synthetic records. No external host or real customer data was accessed.

```
FileNotFoundError: [Errno 2] No such file or directory: 'attack_log.json'
(.venv) ➜  security_governance_lab chmod +x simulate_attack.py
./simulate_attack.py | tee evidence/13-attack-simulation.txt
Simulating attack on vulnerable Struts application...
System is vulnerable to Struts S2-045 (CVE-2017-5638)
Simulating data exfiltration...
Simulating access to database...
Simulating exfiltration of customer data...
Attack simulation completed. Log saved to attack_log.json
(.venv) ➜  security_governance_lab cp attack_log.json evidence/
python3 -m json.tool attack_log.json | tee evidence/14-attack-log-readable.txt
{
    "timestamp": "2026-10-11T15:39:27.753889",
    "attack_type": "Struts S2-045 Exploitation",
    "target": "web_server",
    "success": true,
    "data_accessed": "customer_data database",
    "records_affected": 3
}
(.venv) ➜  security_governance_lab
```

Figure 3 Controlled attack simulation and generated attack log

Governance failure analysis

Category	Observed failure	Impact	Required governance response
Patch management	Critical and high weaknesses remained open	Known exposure remained exploitable	Defined SLA, ownership, escalation and evidence of closure
Incident detection	Successful simulation was not independently detected	Delayed containment	Central monitoring, alerting and response playbooks
Policy implementation	Policies existed without effective enforcement	False assurance	Control testing and accountable owners
Vulnerability management	Critical result persisted	Extended attack window	Risk-based remediation and retesting
Metrics	Measures did not drive action	Poor executive visibility	Thresholds linked to escalation decisions
Security architecture	Segmentation was not demonstrated	Potential lateral movement	Least-privilege network paths and validation
Authentication	Default credentials were accepted	Unauthorized database access	Unique secrets, rotation and restricted administration
Access control	Database exposure was insufficiently constrained	Sensitive-data compromise	Layered application and database authorization

Comparison with the Equifax breach

The simulation closely reflects the governance pattern described in the Equifax case: a known Struts vulnerability remained available for exploitation; monitoring did not stop or independently detect the simulated activity; database access relied on weak credentials; and policy and metrics did not translate into timely corrective action. The laboratory does not reproduce Equifax's scale, duration or real-world harm, but it demonstrates how weak governance permits a technical defect to become an organizational incident.

Governance issue	Equifax case	Laboratory	Similarity
Patch management	Known Struts vulnerability not patched promptly	Vulnerable Struts service remained exposed	High
Monitoring	Breach remained undetected for an	No independent alert blocked the	High

	extended period	simulation	
Segmentation	Attackers could move through the environment	Segmentation could not be confirmed	High
Authentication	Access-control weaknesses contributed to exposure	Default database credentials succeeded	Medium
Policy execution	Policies did not assure implementation	Recorded policies coexisted with open findings	High

Key lessons

A patch policy has value only when responsibilities, deadlines, exceptions and verification are enforced. Monitoring must detect actionable events and route them to an accountable responder. Segmentation should be tested, not assumed. Security metrics should lead to decisions when thresholds are missed. Finally, independent technical verification is essential because a recorded control status may differ from the actual system state.

Part 3 Implementing Security Governance Controls

Controls implemented

Control area	Recorded implementation	Owner or evidence
Patch management	Struts remediation flag and critical patch metric	Security Engineer; CTL-004 and MET-004
Network segmentation	Segmentation flag and control record	Network Engineer; CTL-005
Database security	MySQL security flag and control record	Database Administrator; CTL-006
Security monitoring	Monitoring script and coverage metric	Security Engineer; CTL-007 and MET-005
Governance oversight	Oversight policy and executive control	POL-004 and CTL-008

The supplied patch-management script wrote a new patch_status.json object during each operation. Consequently, the final operation erased the earlier Struts and network values. The combined file was reconstructed only after the terminal output showed that all three simulated operations had completed successfully. This correction repaired the state-recording defect; it did not alter the underlying containers.

Verification results

```
(.venv) → security_governance_lab cp governance_controls_report.md evidence/
./implement_governance_controls.py --verify | tee evidence/19-controls-verification.txt
Vulnerability scan completed. Report saved to vulnerability_report_20261011_155459.json
Summary: 3 vulnerabilities found
  Critical: 1
  High: 1
  Medium: 0
  Low: 0
  Unknown: 1
Verifying security controls...
Struts patched: False
MySQL secured: True
Network segmented: False
Security monitoring script implemented
Governance oversight document implemented
Policies: 4
Controls: 8
Metrics: 5
(.venv) → security_governance_lab python3 - <<'PY'
import json

status = {
    "struts_patched": True,
    "mysql_secured": True,
    "network_segmented": True
}

with open("patch_status.json", "w") as file:
    json.dump(status, file, indent=2)

print(json.dumps(status, indent=2))
PY
{
  "struts_patched": true,
  "mysql_secured": true,
  "network_segmented": true
}
(.venv) → security_governance_lab cp patch_status.json evidence/
./implement_governance_controls.py --verify | tee evidence/20-controls-verification-corrected.txt
Vulnerability scan completed. Report saved to vulnerability_report_20261011_155635.json
Summary: 3 vulnerabilities found
  Critical: 1
  High: 1
  Medium: 0
  Low: 0
  Unknown: 1
Verifying security controls...
Struts patched: True
MySQL secured: True
Network segmented: True
Security monitoring script implemented
Governance oversight document implemented
Policies: 4
Controls: 8
Metrics: 5
(.venv) → security_governance_lab ./simulate_attack.py | tee evidence/21-post-control-attack-test.txt
Simulating attack on vulnerable Struts application...
System is vulnerable to Struts S2-045 (CVE-2017-5638)
Simulating data exfiltration...
Simulating access to database...
Simulating exfiltration of customer data...
Attack simulation completed. Log saved to attack_log.json
(.venv) → security_governance_lab
```

Figure 4 Corrected control status followed by an unsuccessful technical assurance outcome

Verification layer	Outcome	Interpretation
Administrative status	Struts, MySQL and segmentation flags true	The simulation recorded intended completion
Artefact existence	Monitoring and oversight files present	Required governance records were created
Vulnerability scan	Critical and high findings persisted	Underlying technical exposure was not remediated
Attack retest	Simulation remained successful	Recorded status was not evidence of operating effectiveness

The correct audit conclusion is therefore qualified: the controls were designed and recorded, but their operating effectiveness was not demonstrated. Closure should require replacement of the vulnerable Struts component, credential changes, protocol-appropriate segmentation testing and a clean retest. Until then, the critical risk remains open.

Part 4 Security Governance Metrics and Reporting

Metrics framework

The framework covers implementation, effectiveness, efficiency and business impact. Each metric should have a definition, formula, source, target, reporting frequency, owner and audience. Results should follow a lifecycle of define, collect, analyze, report, act and review.

Metric	Purpose	Target	Primary audience
Patch compliance	Percentage of critical systems patched within SLA	At least 95%	CISO and executive management
Remediation time	Average days to close critical vulnerabilities	7 days or less	Security and IT leadership
Security incidents	Number of confirmed incidents	0	Executives and Board
Monitoring coverage	Critical systems under active monitoring	At least 95%	SOC and CISO
Policy compliance	Policies implemented and operating effectively	At least 95%	Compliance and Board

Trend reporting

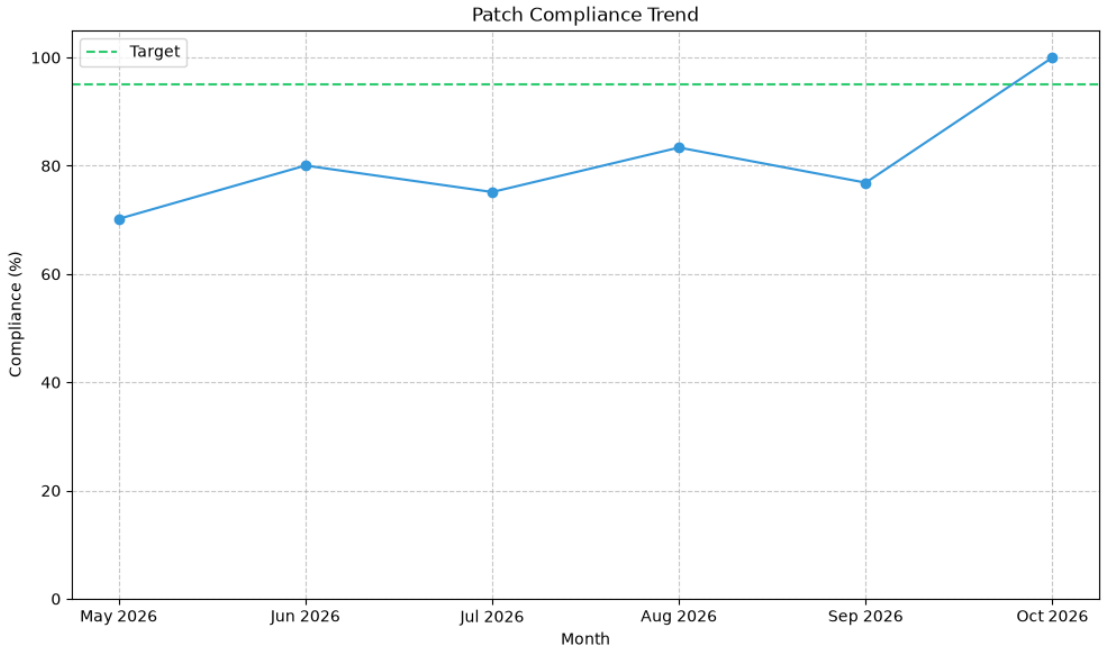


Figure 5 Patch compliance trend

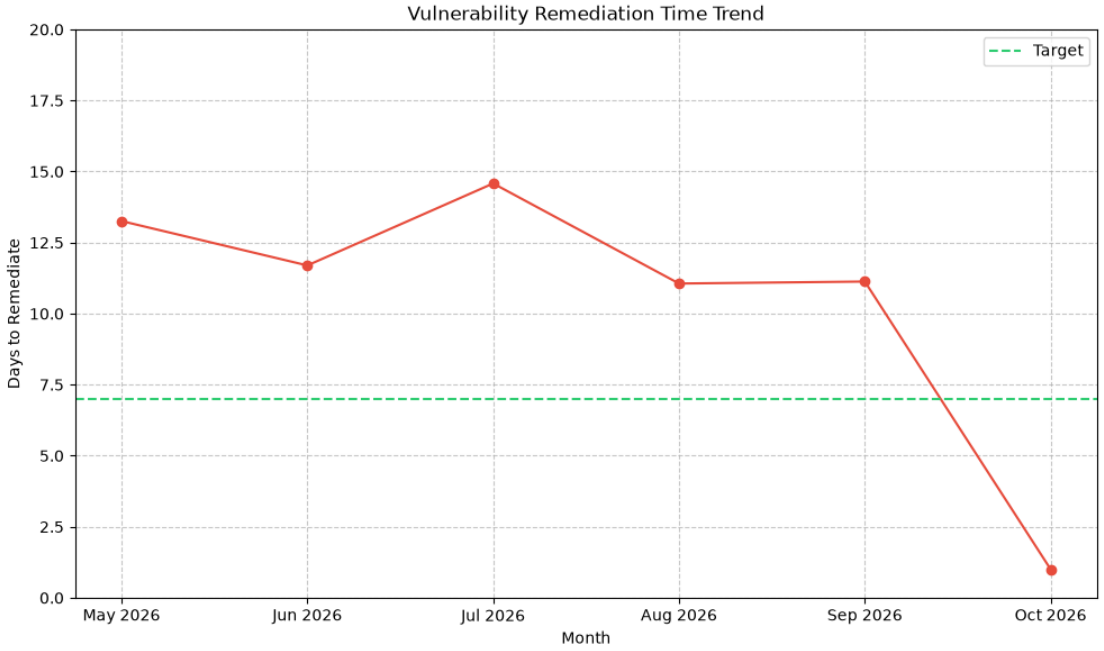


Figure 6 Vulnerability remediation trend

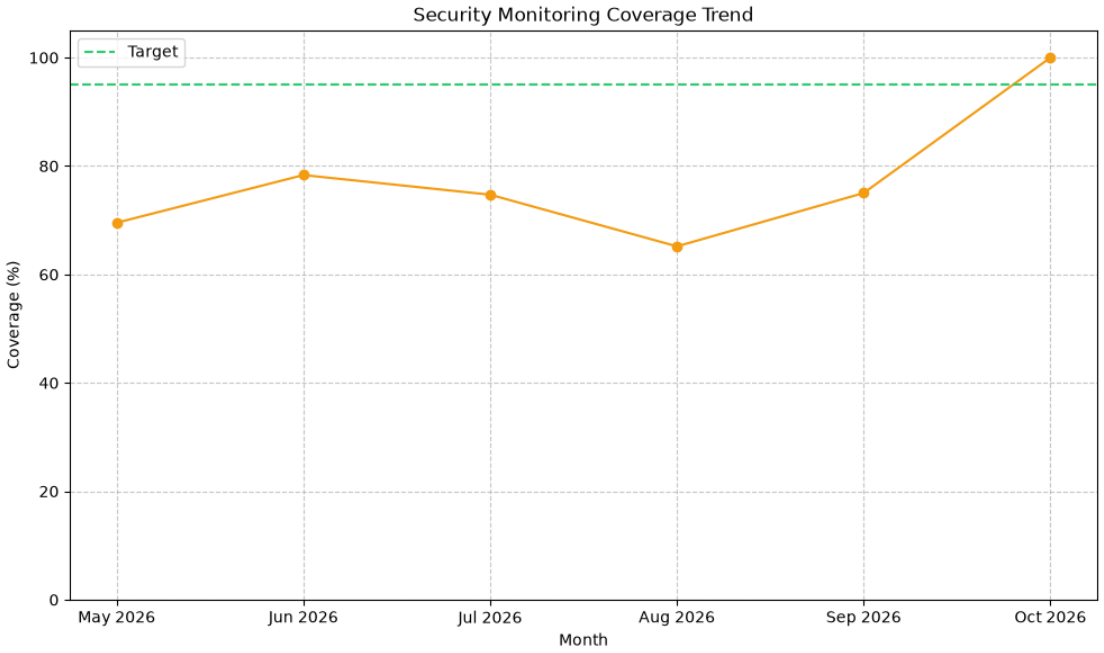


Figure 7 Security monitoring coverage trend

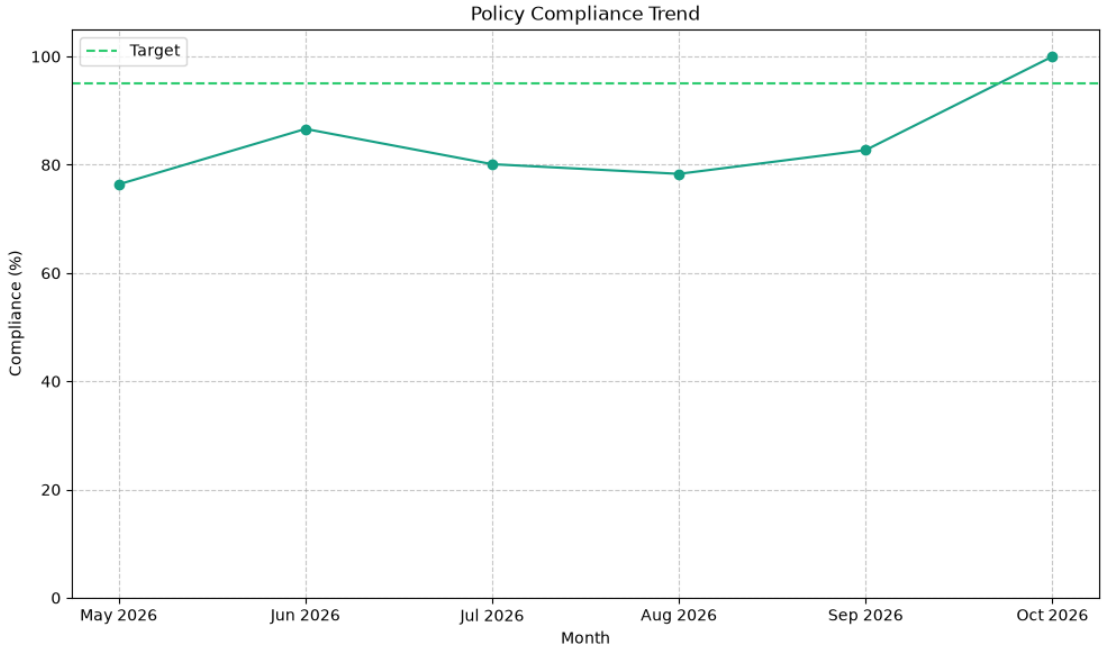


Figure 8 Policy compliance trend

Use of the executive dashboard

Executives should use red and yellow indicators to trigger decisions rather than treating the dashboard as passive reporting. Trends provide context, but evidence of technical retesting must accompany claims that critical risk has been reduced. The dashboard should be refreshed monthly and immediately after a material incident or major control change.

Conclusion and Recommendations

The exercise demonstrates that the main security failure was not merely an outdated software component. Governance weaknesses allowed a known vulnerability, weak credentials and uncertain segmentation to coexist with policies and metrics. The successful post-control attack test confirms the importance of separating implementation records from operating-effectiveness evidence.

- Replace the vulnerable Struts component with a supported, verified version and rerun the scan and attack test.
- Change all default database credentials, store secrets securely and restrict administrative access.
- Implement and test network rules that permit only required application-to-database traffic.
- Require evidence-based closure for critical findings, including an independent retest.
- Correct the patch-status logic so each control update merges with existing state rather than overwriting it.
- Escalate missed critical-patch and remediation-time targets to executive management and the Board risk committee.
- Retain scan reports, tickets, approvals, exceptions and verification results as audit evidence.

Evidence Register

Evidence	Purpose
Container status, logs and HTTP response	Establishes the operational baseline
Vulnerability reports	Records initial and verification findings
Attack log	Records the controlled simulated impact
Governance tracker dashboard	Shows policies, controls, risks, incidents and metrics
Governance failure report	Documents eight failure categories
Control implementation and verification outputs	Separates recorded status from technical effectiveness
Executive dashboard, board report and charts	Supports management and Board oversight

References

1. ICDFA. GRC102 Practical Lab 8 Laboratory Instructions. Supplied assessment material, 2026.
2. National Institute of Standards and Technology. CVE-2017-5638 Detail. National Vulnerability Database.
3. U.S. Government Accountability Office. Data Protection: Actions Taken by Equifax and Federal Agencies in Response to the 2017 Breach. GAO-18-559, 2018.
4. Federal Trade Commission. Equifax to Pay $575 Million as Part of Settlement with FTC, CFPB, and States Related to 2017 Data Breach. 2019.
5. Center for Internet Security. CIS Critical Security Controls: Network Infrastructure Management and Network Monitoring and Defense.